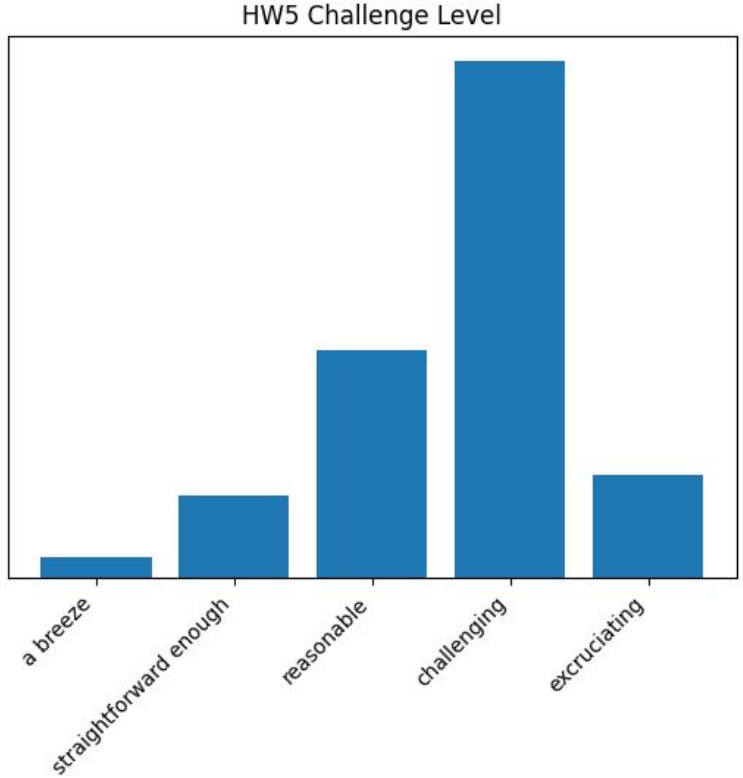
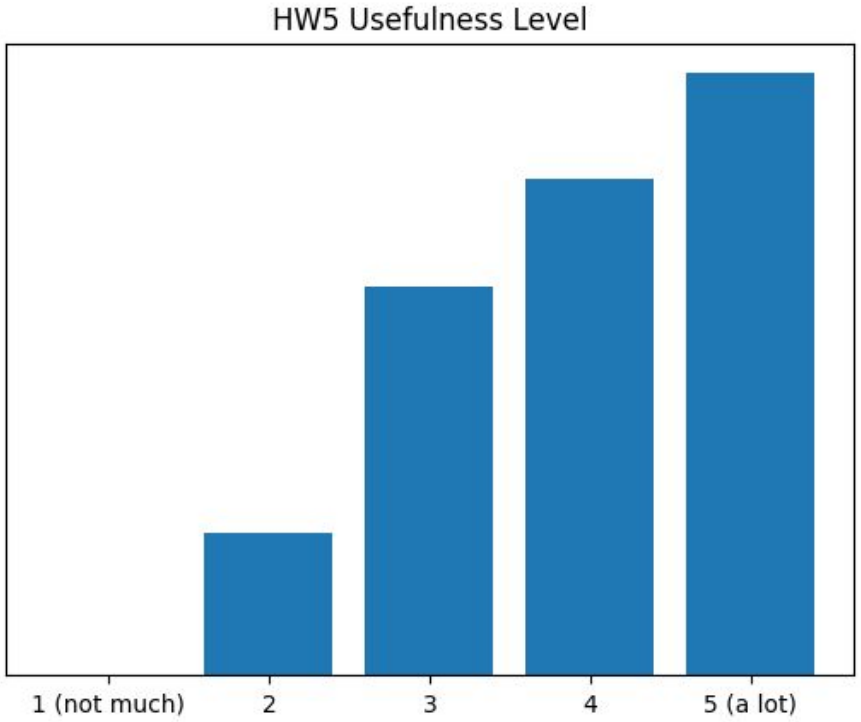
CSE 493L Section 7

Cache Coherence & Misc Topics

Looking Back on Homework 5

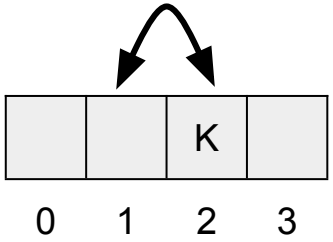




HW 6 Cuckoo - **Hard Deadline Tomorrow**

- New tests have been added to the autograder.
  - If necessary, more tests may be added after the due date.
  - We try to avoid this by making the initial tests as helpful as possible.
  - Suggestion: paste the contents of hw6-student-tests into **tests**.
- Loom won't check interleavings unless you use: **RUSTFLAGS="--cfg loom" cargo test --test loom_tests --release**
- Don't worry! It's expected that many submissions won't get 100%. Concurrency is hard. You're all doing really great!

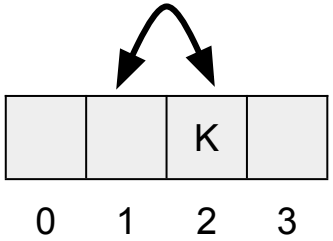
HW 6 Cuckoo - Tips



(Sorry for not getting to all the Cuckoo stuff last section.)

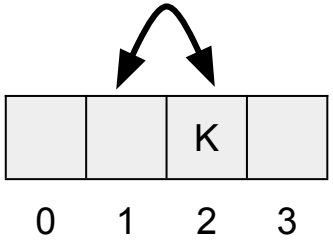
- Other threads may be moving elements between buckets.
- All you can assume is that an element will be in one of its 2 buckets.
- Why should **lookup()** lock both buckets before checking?

HW 6 Cuckoo - Tips



- Suppose we lock 1, then 2. Another thread locks 2, then 1. Deadlock!
  - Solution: **ordered locking**.
- Rule: never try to lock a lower index while you're holding a higher index.
  - For simplicity, consider holding no more than 2 locks at a time.
  - Can create helper function that acquires 2 locks in order.

HW 6 Cuckoo - Tips



When moving an element along an insertion path:

- Lock both the source (full) and destination (empty) buckets.
  - Otherwise, another thread might insert into the destination, leaving both buckets full. Now your element doesn’t have a home!
- Confirm the destination is really one of the 2 buckets for this element.
  - Another thread might have put a different element into the source bucket.
  - If the destination isn’t a valid bucket, the insertion path is no longer valid, so return an error.

Homework 7: Due Dates

- **Monday, February 23rd:** test case - hard deadline
- **Wednesday, February 25th**: code, questions - 2 day grace period

Implementing a cache coherence simulator.

HW 7 - Cache Coherence Simulator

Simulated directory-based MSI system.

- Devices can send each other messages through simulated “hardware” FIFOs.
  - Unlike in distributed systems, messages cannot be dropped or reordered.
  - Each message is sent only once, so the devices must put them in internal queues if responding immediately isn’t possible.
- You will implement a simplified memory controller.
  - Must reply to cache requests, and send invalidate requests as necessary.
  - Read the readme early!

(Code walkthrough time)

Memory Consistency Review

A core reorders assembly instructions for performance.

- This reordering won't affect behavior when only one core is running.
- However, it does affect multi-core behavior!
- Different architectures make different promises about what they won't reorder.
- Memory fence prevents instructions from being reordered across it.

```
instruction 1
instruction 2

memory fence

instruction 3
instruction 4
```

Memory Consistency Lecture Example

```
write "hi" to data

write true to flag
```

```
loop {
  read flag to reg1
  if reg1 == true { break }
}

read data to reg2
assert(data == "hi")
```

Memory Consistency Lecture Example

```
write "hi" to data
MEMORY FENCE
write true to flag
```

```
loop {
  read flag to reg1
  if reg1 == true { break }
}

read data to reg2
assert(data == "hi")
```

Memory Consistency Lecture Example

```
write "hi" to data
MEMORY FENCE
write true to flag
```

```
loop {
  read flag to reg1
  if reg1 == true { break }
}
MEMORY FENCE
read data to reg2
assert(data == "hi")
```

Demos

Link: https://gitlab.cs.washington.edu/cse334-26wi/slides

- Monomorphization vs dynamic dispatch vs enums.
- False sharing.

A note on `mut`

Why does Rust disallow multiple mutable borrows in single threaded programs?

A note on `mut`

Why does Rust disallow multiple mutable borrows in single threaded programs?

- Using multiple mutable borrows can still lead to undefined behavior.

Iterator Invalidation

```
fn foo() {
    let mut v = Vec::from([1, 2, 3, 4, 5]);
    let v_ref = &mut v;

    for e in &mut v {
        v_ref.push(100);
        println!("{e}");
    }
}
```

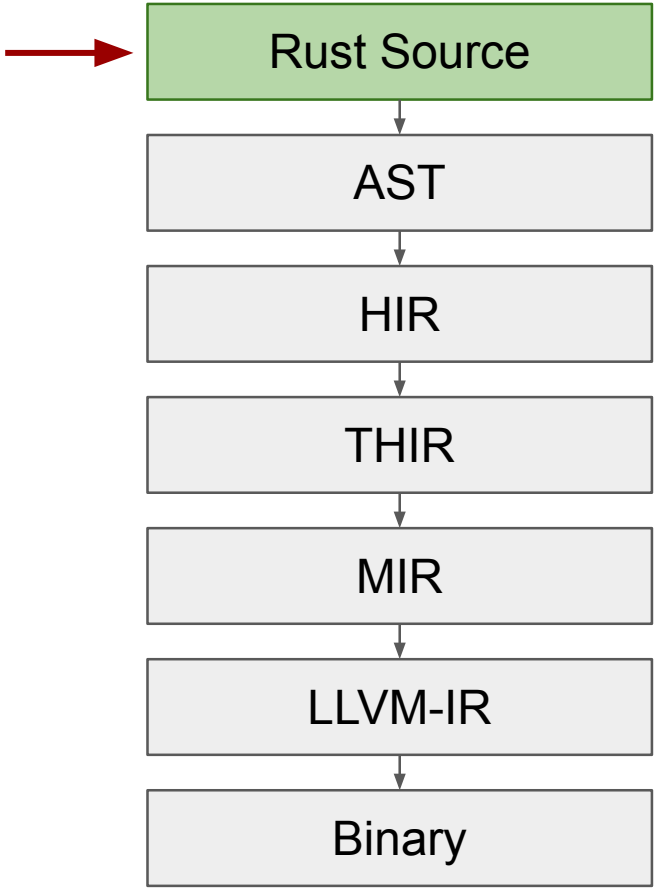
`Vec::push` could reallocate, leaving `e` to be a dangling…

A note on `mut`

The compiler treats `mut` as meaning ***exclusive***

- Allows codegen optimizations!

```rust
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```



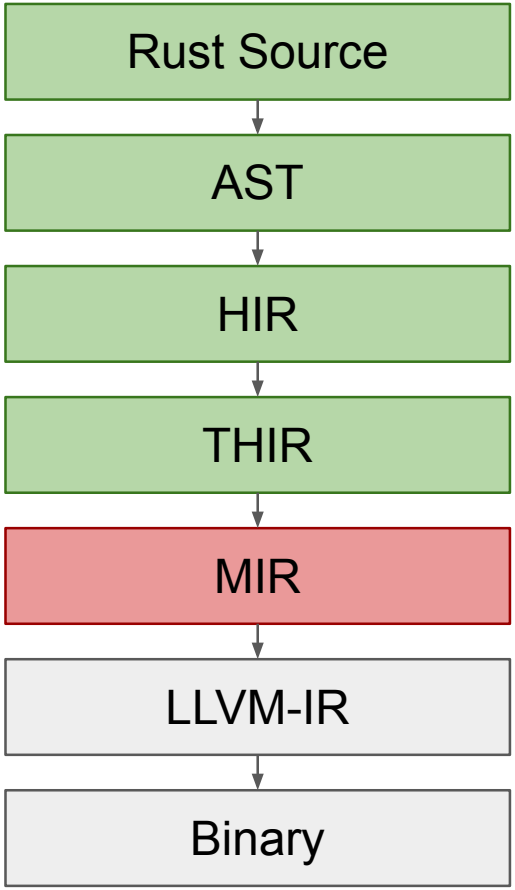
A note on `mut`

The compiler treats `mut` as meaning ***exclusive***

- Allows codegen optimizations!

```
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::wit
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

I can't compile this! `bad_ref` is not an exclusive reference!

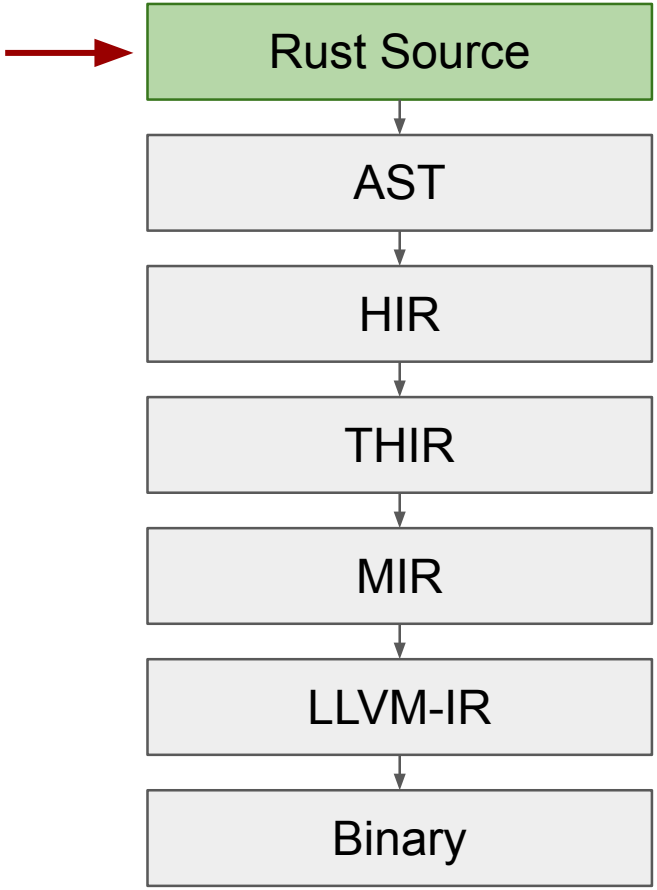


A note on `mut`

The compiler treats `mut` as meaning ***exclusive***

- Allows codegen optimizations!

```rust
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());

    s.push_str(a);
    s.push_str(b);

    println!("{}", s);
}
```

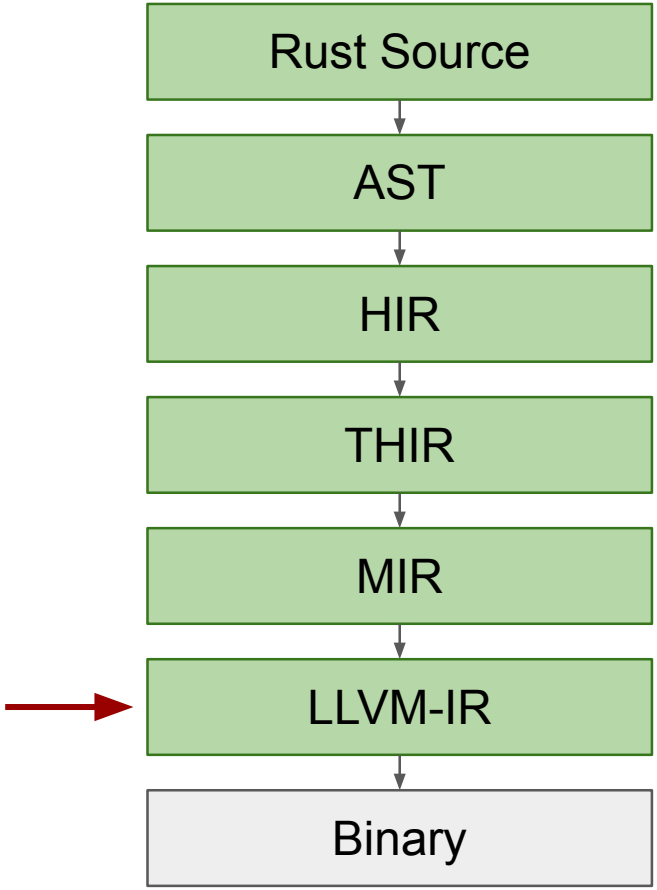


A note on `mut`

The compiler treats `mut` as meaning ***exclusive***

- Allows codegen optimizations!

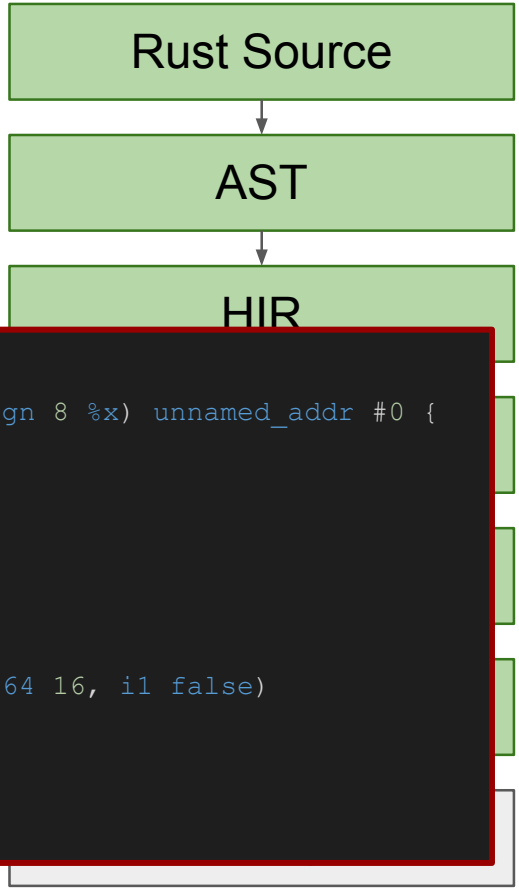
```rust
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());

    s.push_str(a);
    s.push_str(b);

    println!("{}", s);
}
```



A note on `mut`

The compiler treats `mut` as meaning ***exclusive***

- Allows codegen optimizations!




```
...
define internal void @...main(ptr sret([16 x i8]) align 8 %_0, ptr align 8 %x) unnamed_addr #0 {
start:
  %_2 = alloca [16 x i8], align 8
  store ptr %x, ptr %_2, align 8
  %0 = getelementptr inbounds i8, ptr %_2, i64 8
  store ptr @...main, ptr %0, align 8
  call void @llvm.memcpy.p0.p0.i64(ptr align 8 %_0, ptr align 8 %_2, i64 16, i1 false)
  ret void
}
...
```

A note on `mut`

LLVM-IR uses a `noalias` attribute to mark pointers as exclusive

- Allows for heavier reordering of instructions to dispatch memory ops earlier
  - e.g. hoisting reads
- Rust's mutable references can take heavy advantage of this!
  - Uses MIR checks to guarantee mutable references are exclusive, then codegen can use this fact to optimize!

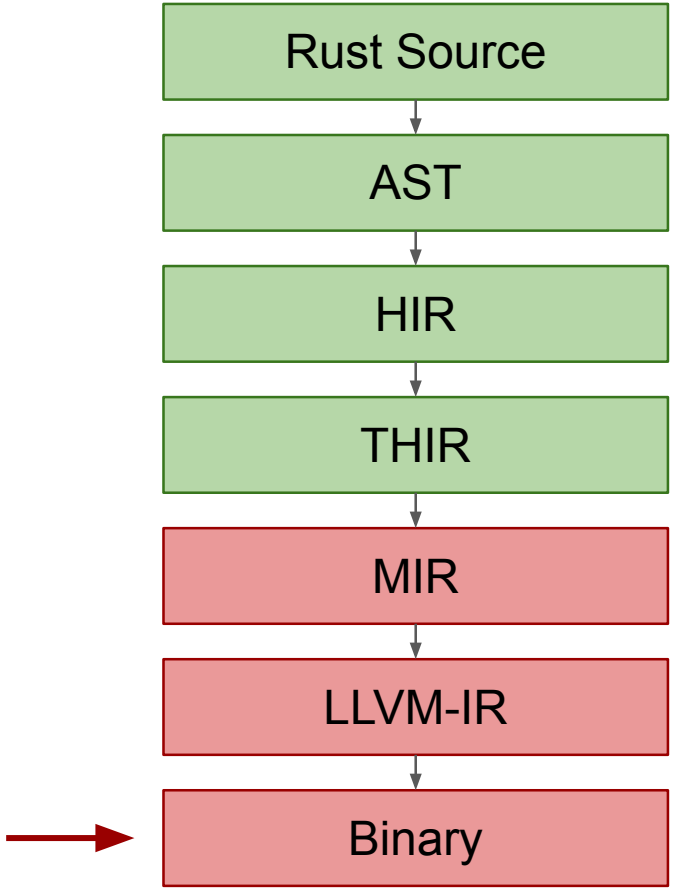
```
declare void @llvm.memcpy.p0.p0.i64 (
    ptr noalias writeonly captures(none),
    ptr noalias readonly captures(none),
    i64,
    i1 immarg
)
```

A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```rust
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```



A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

`println!()` requires me to interpolate `bad_ref` into "`{}`"

A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

I need `bad_ref.ptr` and `bad_ref.length` to interpolate

A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

IO takes time! I should issue both reads early!

A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```rust
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

`bad_ref` isn't used to mutate anything in this range…

A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

!!!

...and I was told that `bad_ref` is exclusive, so nothing else could have mutated it!

A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

Which means I can issue the read for the length here!*

* The memory read could even disappear, if a constant is discovered and propagated…

A note on `mut`

What if we did allow for multiple mutable references?

- LLVM-IR expects that `bad_ref` is exclusive

```
fn main() {
    let a = "Hello, ";
    let b = "world!";

    let mut s = String::with_capacity(a.len() + b.len());
    let bad_ref = &mut s;

    s.push_str(a);
    s.push_str(b);

    println!("{}", bad_ref);
}
```

This is obviously wrong! At this point, `s`'s length is 0!

Which means I can issue the read for the length here!*

* The memory read could even disappear, if a constant is discovered and propagated…

*Poly*morphism vs *Mono*morphism

Remember this HW4 question? A lot of people provided an answer using traits…

You are designing a type model for a graphing calculator, which can use Polar or Cartesian coordinates to represent 2 dimensional points. You have a bunch of functions which operate on these points (one to draw a line between two of them, another to draw a square, etc...); these functions should work regardless of the coordinate system being used to define any individual point. Management has told you to expect needing to extend the number of different coordinate types in the future.

How would you define the Point type? What are the benefits of your choice?

***Poly*morphism** vs *Mono*morphism

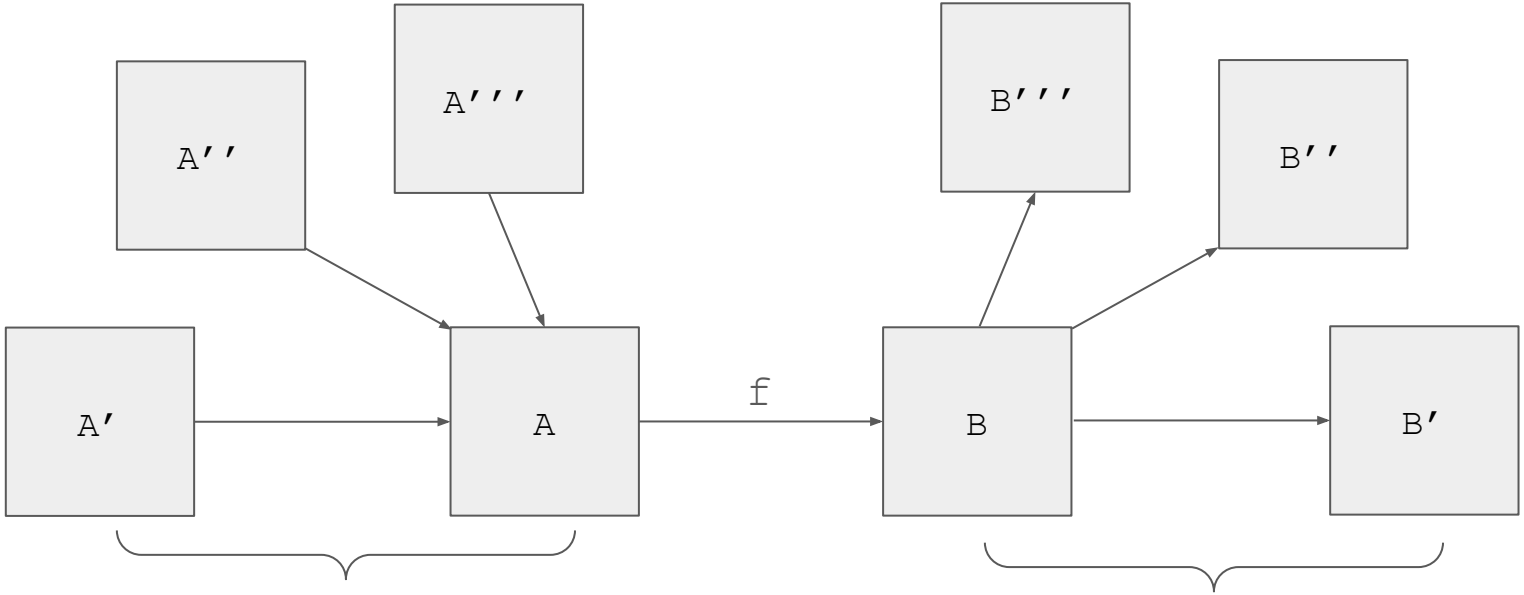
Statically known types are powerful and allow for deeper optimization.

We'll talk about this abstractly, `f` here could represent any operation

(e.g. `horizontal shift(point, distance)`)

***Poly*morphism** vs *Mono*morphism

Statically known types are powerful and allow for deeper optimization.

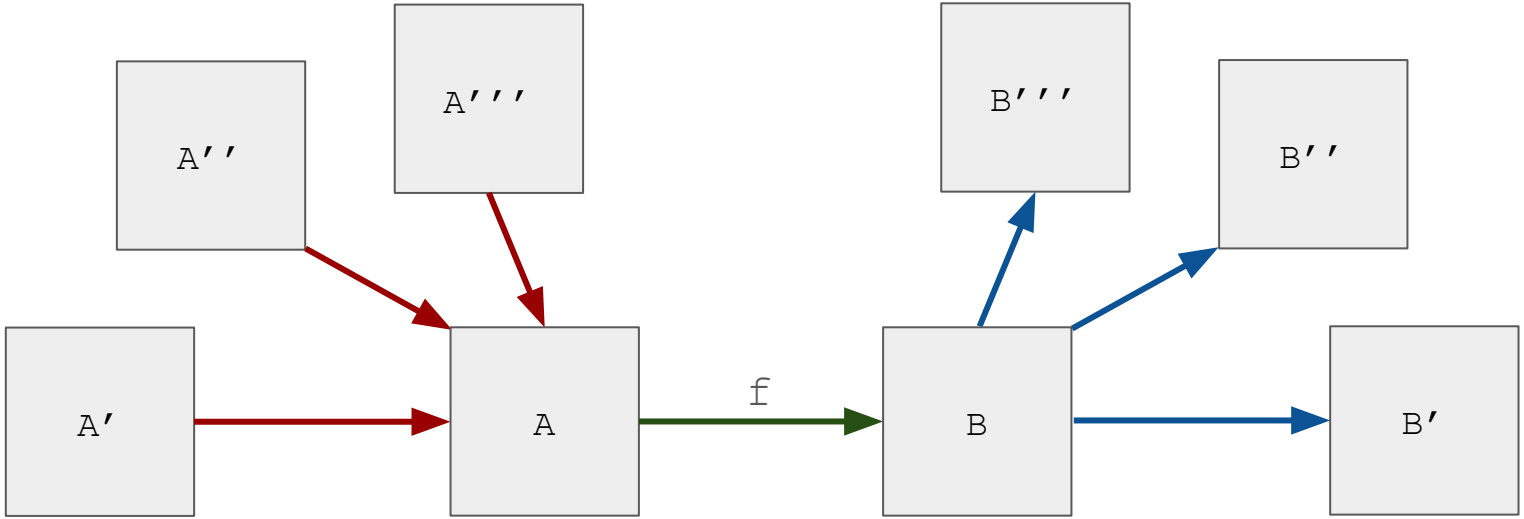


Convert to canonical representation
(e.g. Polar → Cartesian)

Convert from canonical representation
(e.g. Cartesian → Polar)

***Poly*morphism** vs *Mono*morphism

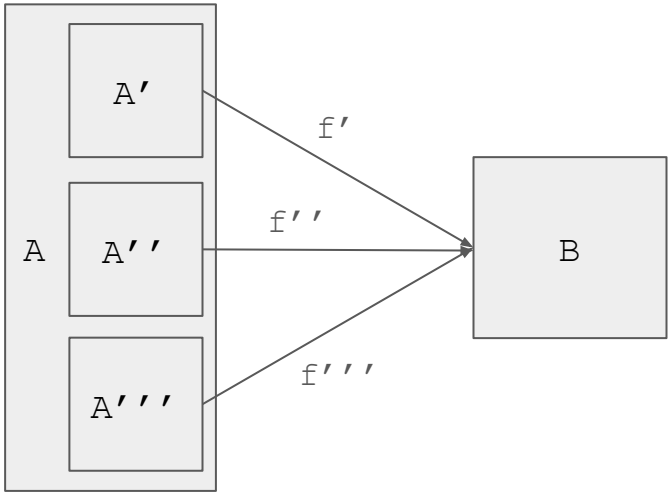
Statically known types are powerful and allow for deeper optimization.



A’ → B’ cannot be optimized altogether, each arrow has to be done separately

*Poly*morphism vs ***Mono*morphism**

Statically known types are powerful and allow for deeper optimization.



Full A’ → B path can be optimized as a single operation

Intuitively, rotating a point in polar form is really easy, *so why convert to cartesian to do it*?

*Poly*morphism vs *Mono*morphism

Static dispatch is generally much better than dynamic dispatch.

- Allows for deeper optimization with all types known at compile time
- Does not require vtable lookups to determine which function to run
  - Pattern matching enums compiles down to branches (<4 cases) or jump tables, both of which are generally faster than vtable dereferences
- Exhaustive pattern matching also means it’s easy to find places where code updates are necessary

Use dynamic dispatch only if you don’t statically know the list of variants.

- e.g. you are designing a library that expects consumers to define their own data types that somehow interact with your library.
- In general though, try to avoid trait objects